```
// alloc_buffer
%smemSP = memref.alloc() {alignment = 16 : i64} : memref<4x128x128xf32>
%regSP = memref.alloc() {alignment = 16 : i64} : memref<64xf32>
...
affine.for %bk = 0 to 32 {  // Local Split K (128 to 32)
  affine.for %i = 0 to 16 {
    %0 = affine.load %smemA[(%tid floordiv 128) * 32 + %bk, map1(%tid, %i)] : memref<128x8xf32>
    affine.store %0, %regA[%i] : memref<16xf32>
  }
  affine.for %i = 0 to 16 {
    %0 = affine.load %smemB[(%tid floordiv 128) * 32 + %bk, map2(%tid, %i)] : memref<128x8xf32>
    affine.store %0, %regB[%i] : memref<16xf32>
  }
  affine.for %i = 0 to 16 {
    affine.for %j = 0 to 16 {
      %0 = affine.load %regA[%i] : memref<16xf32>
      %1 = affine.load %regB[%j] : memref<16xf32>
      %2 = affine.load %RegC[%i, %j] : memref<16x16xf32>
      %3 = arith.mulf %0, %1 : f32
      %4 = arith.addf %2, %3 : f32
      affine.store %4, %RegC[%i, %j] : memref<16x16xf32>
    }
  }
}
// Store to "SM(SP)"
affine.for %i = 0 to 16 {
  affine.for %j = 0 to 16{
    %0 = affine.load %RegC[%i, %j] : memref<16x16xf32>
    affine.store %0, %smemSP[(%tid floordiv 64), map1(%tid, %i), map2(%tid, %j)] : memref<4
    x128x128xf32>
  }
}
gpu.barrier
// Add
affine.for %i = 0 to 64 {
  %0 = affine.load %smemSP[0, map3(%tid, %i)] : memref<4x128x128xf32>
  affine.store %0, %regSP[%i] : memref<64xf32>
}
affine.for %i = 1 to 4 {
  affine.for %j = 0 to 64 {
  %0 = affine.load %smemSP[%i, map3(%tid, %j)] : memref<4x128x128xf32>
  %1 = affine.load %regSP[%j] : memref<64xf32>
  %2 = arith.addf %0, %1 : f32
  affine.store %2, %regSP[%j] : memref<64xf32>
  }
}
```